PARALLEL OCR PIPELINE

CSC611M | Group 3

November 7, 2026

01

Problem

The Problem

When digitizing large volumes of documents (e.g., invoices, IDs, or forms), traditional OCR systems process images **sequentially**:

Load Image → **Preprocess** → **Run OCR Inference** → **Save Result**

Meanwhile, on a modern machine with 8 or 12 CPU cores, a sequential program uses **only 1 core**, leaving the other **7–11 cores completely idle**.

If an image takes **~100 ms** to process:

1,000 images ≈ 1.6 minutes
100,000 images ≈ 2.7 hours

02

Strategy

THE STRATEGY

The goal is to **cut document processing time** by running images across all CPU cores simultaneously, while ensuring the shared output file remains **accurate**, correctly ordered, and deadlock-free.

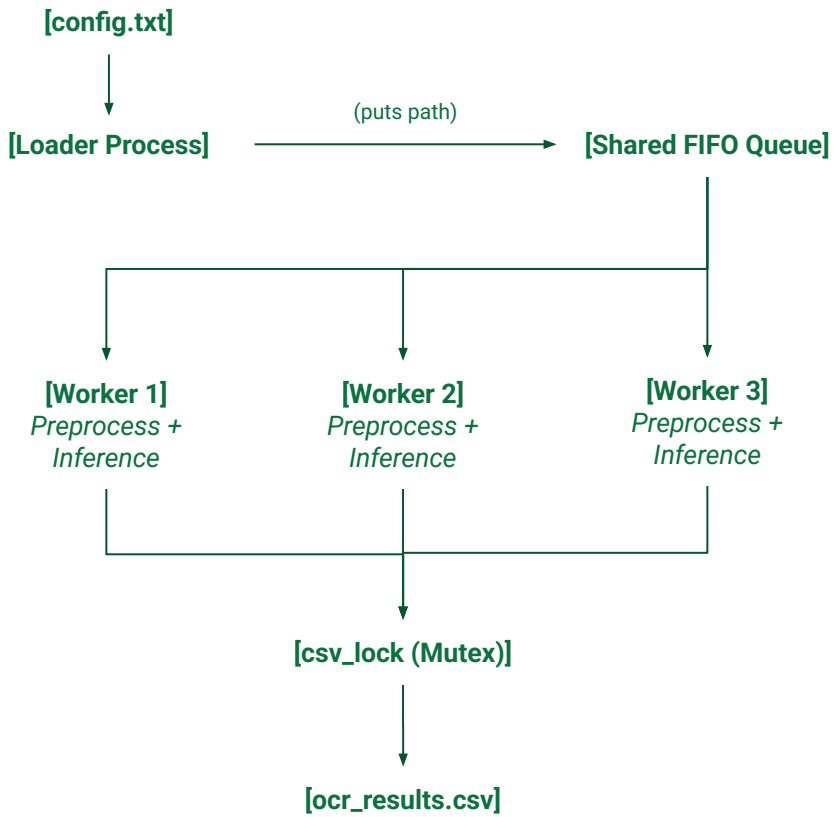
Python GIL	• **threading.Thread** serializes thread execution during CPU-heavy computing. • use **multiprocessing.Process**, giving each worker its own Python interpreter and core.
Producer-Consumer	• a worker stuck with complex/noisy images lags behind (**load imbalance**) • Loader (Producer) streams images into a shared FIFO Queue, and Workers (Consumers) pull images **dynamically**
Race Conditions	• simultaneous writes collide, corrupting rows and duplicating IDs • **mutual exclusion lock** (Lock) creates a protected critical section around the atomic counter and file append
Clean Termination	• checking an empty queue too early might quit; waiting forever may cause deadlocks. • Poison Pill (**Sentinel**) pattern. When the loader is done, it drops a **None** token for each worker. When a worker reads None, it exits gracefully.

03

Pipeline

PIPELINE ARCHITECTURE

1. **The Producer (Loader Process):**
   - Reads config.txt dynamically at runtime.
   - Scans dataset/ and **pushes image paths** into **multiprocessing.Queue**.
   - Sends n None sentinel tokens (poison pills) when finished.
2. **The Consumers (Worker Pool):**
   - Explicitly spawns n **independent processes** across CPU cores.
   - Pulls tasks dynamically → Runs **Grayscale/Denoise/Contrast/Otsu** preprocessing → Extracts text with OCR.
3. **Synchronized Output:**
   - Protected by **csv_lock** to ensure atomic sequential IDs (1, 2, 3...) and prevent corrupted rows.



Preprocessing



0. ORIGINAL



1. GRAYSCALE

Collapses 3-channel RGB into a 2D intensity matrix.



2. DENOISE

Removes salt-and-pepper noise and speckles while keeping character edges crisp.



3. CONTRAST

Widens the difference between faint text and paper backgrounds.



4. OTSU BINARIZATION

Converts pixels to pure black and white for the OCR engine.

04

Benchmarking

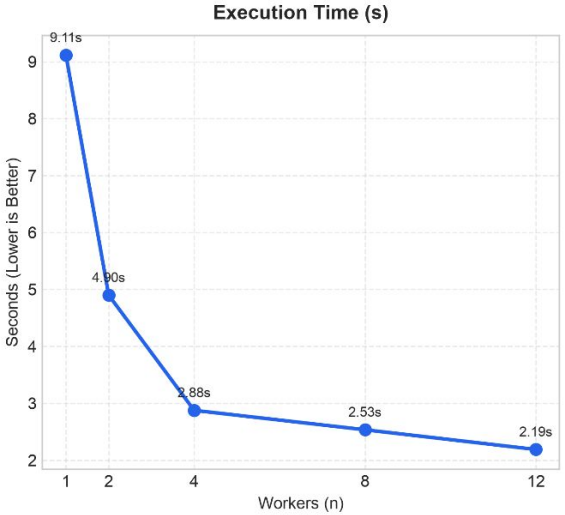
BENCHMARKING

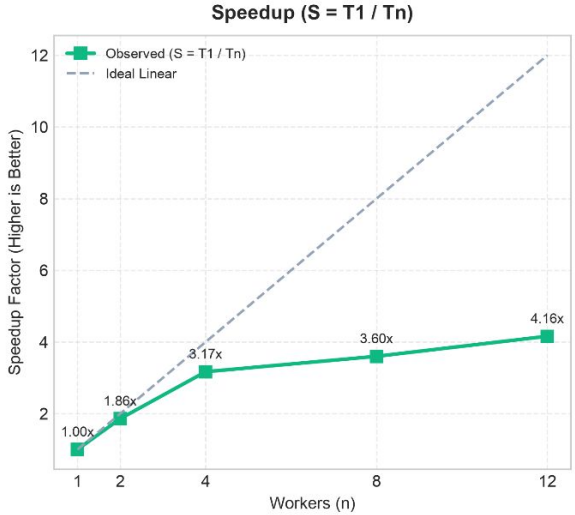
- **Speedup Formula:** $S = T_1 / T_n$ *(Where T_1 is baseline time with 1 worker, and T_n is total time with n workers)*
- **Efficiency Formula:** $E = S / n \times 100\%$
- **Throughput Formula:** $X = \text{Total Images} / T_n$

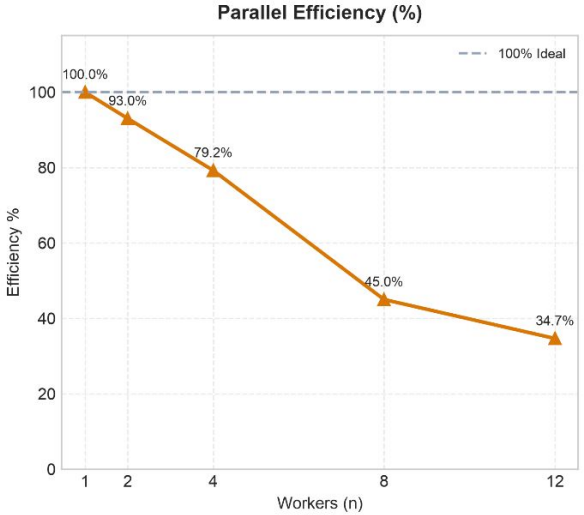
Workers (n)	Time (T_n)	Speedup (S)	Efficiency	Throughput
1	9.1128	1.0	100.0	10.97
2	4.8985	1.86	93.0	20.41
4	**2.876**	**3.17**	**79.25**	**34.77**
8	2.5345	3.6	45.0	39.46
12	2.1882	4.16	34.67	45.7

SPEEDUP ANALYSIS

- **Speedup Formula:** $S = T_1 / T_n$ *(Where T_1 is baseline time with 1 worker, and T_n is total time with n workers)*
- **Efficiency Formula:** $E = S / n \times 100\%$







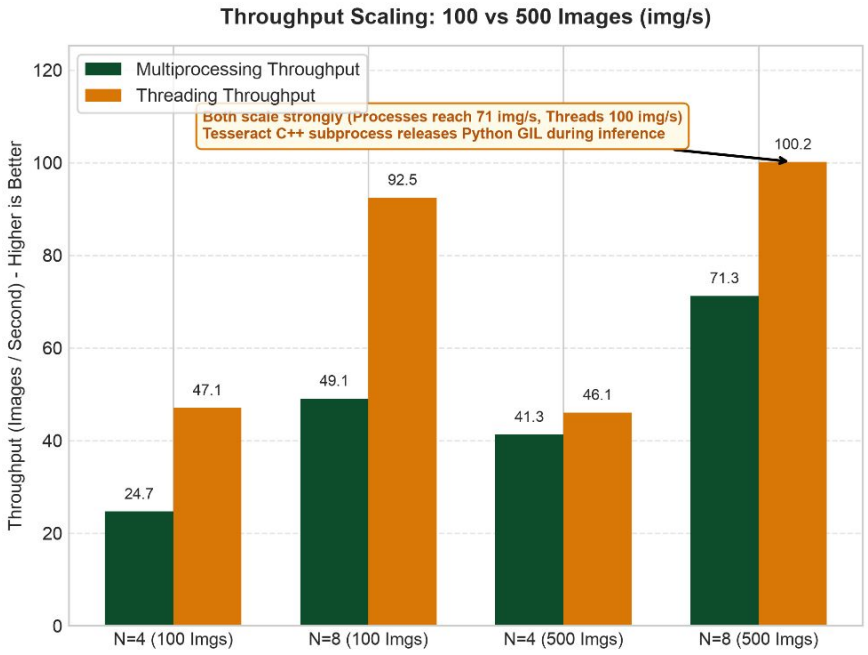
Execution time **drops sharply from 9.11s down to 2.88s at 4 workers** (a 68% reduction), before leveling off as additional workers yield minimal latency gains.

Speedup **scales near-linearly up to 4 workers (3.17x)**, after which the curve flattens toward 4.16x at 12 workers due to Amdahl's Law.

Parallel efficiency remains high at **79.2% for 4 workers**, but plummets to 34.7% at 12 workers as serial queue and process overhead dominate.

PROCESSES VS THREADS

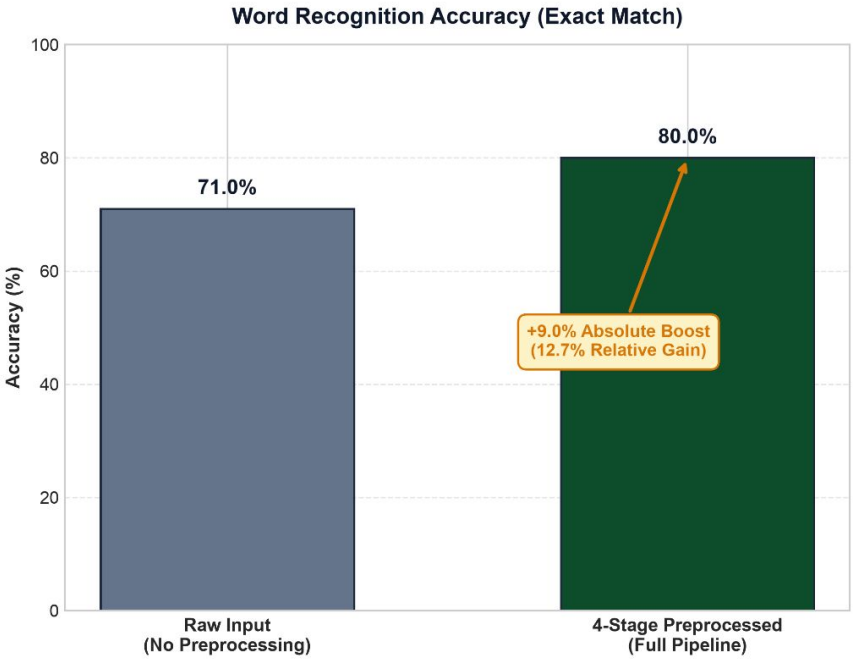


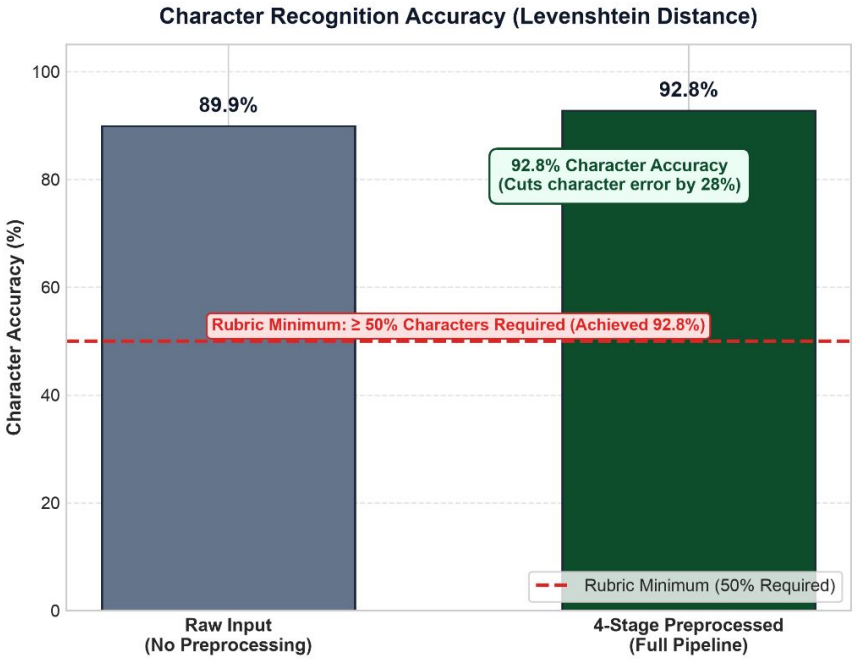


Threads finish roughly 1.9× faster on the 100-image dataset (1.08s vs 2.04s at 8 workers) because they eliminate Python interpreter spawn latency and IPC pipe overhead.

Both backends scale strongly as the workload grows, with processes reaching 71.3 img/s and threads hitting 100.2 img/s, since Tesseract's native C++ subprocess releases the Python GIL during inference.

ACCURACY





+9.0% Exact Match Boost: Full-word recognition rose from 71.0% (Raw) to 80.0% (Preprocessed), a 12.7% relative improvement across the dataset.

92.8% Character Accuracy: Computed via Levenshtein edit distance (1–CER), cutting total character error rate by 28.4% (89.9% → 92.8%).

05

Optimization

OPTIMIZATION

01 Vectorized Otsu Binarization

Replaced slow Python for loops with fast NumPy operations, calculating the threshold in milliseconds.

02 In-Memory Pipeline

Images stay entirely in RAM from preprocessing to OCR, completely eliminating slow disk reads and writes.

03 Process Spawning Overhead

Workers are created once at startup instead of per image, avoiding Python's ~500ms startup delay.

04 Lock Contention

The lock is held for only microseconds to write results, so workers never get stuck waiting on each other.

05 Amdahl’s Law

4 workers cleanly match available CPU cores, maximizing speedup before Amdahl's Law causes diminishing returns.

Questions?

THANK YOU

GROUP 3